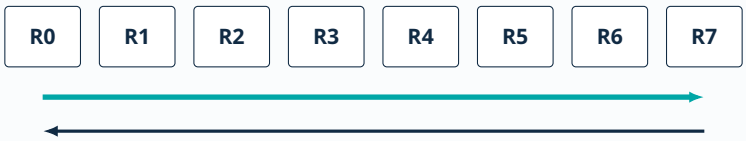
HIGH-PERFORMANCE

16-BIT ARCHITECTURE

LIM M2

Processor Architecture Reference Manual

A deterministic, high-frequency 16-bit microprogrammed processor architecture featuring conflict-free multiplexer routing and 24-bit extended addressing.



POINT-TO-POINT DATA ROUTING · ZERO BUS CONTENTION

OFFICIAL REFERENCE SPECIFICATION

Revision 0.2 — October 2026

LIM M2 Core Architecture Manual. All rights reserved.

Contents

1 Document Scope and Architectural Philosophy **3**
1.1 Notational Conventions . . . 3
1.2 Unified Instruction Syntax Philosophy . . . 3

2 Processor Overview **3**
2.1 Conceptual Datapath Schematic . . . 5

3 Architecturally Visible Registers **6**
3.1 Flag and Status Register (FREG) Mapping . . . 6

4 Machine Instruction Format **7**
4.1 Universal Instruction Word Format . . . 7
4.2 Class I: Single-Byte Instructions (1 Byte, 0 Extra Octets) . . . 7
4.3 Class II: Three-Operand Arithmetic Instructions (2 Bytes) . . . 7
4.4 Class III: Two-Operand Logic Instructions (2 Bytes) . . . 8
4.5 Class IV: Register-to-Register Move (LRR, 2 Bytes) . . . 8
4.6 Class V: Hardware Comparator (CMP, 2 Bytes) . . . 8
4.7 Class VI: Iterative D-Flip-Flop Hardware Shift (BSL, BSR) . . . 8
4.8 Class VII: Stack Operations (PUSH, POP, 2 Bytes) . . . 9
4.9 Class VIII: Special Register Manipulation (MSP, Opcode 26) . . . 9
4.9.1 Format 8.1: FREG Load (MOD = 0, REG = 00) . . . 9
4.9.2 Format 8.2: FREG Bitwise Logic (MOD = 1, REG = 00) . . . 9
4.9.3 Format 8.3: PTREG & SREG Pointer Load (MOD = 0, REG = 01 / 10) . . . 10
4.9.4 Format 8.4: PTREG & SREG Pointer Arithmetic (MOD = 1, REG = 01 / 10) . . . 10
4.9.5 Format 8.5: Vector Registers Load (MOD = 0, REG = 11) . . . 10
4.10 Class IX: Branch Instructions (2–3 Bytes) . . . 11
4.11 Class X: Memory Transfer Instructions (LMR, LRM, 2–4 Bytes) . . . 11

5 Instruction Set Architecture Summary **11**

6 Detailed Instruction Specifications **12**
NOP — Hardware No-Operation and Pipeline Flush . . . 13
ADD — Addition with Automatic Multi-Word Carry Chaining . . . 15
SUB — Subtraction with Automatic Multi-Word Borrow Chaining . . . 17
MUL — Multiplication with Upper Word Residual Retention . . . 19
DIV — Division with Remainder Latched in ALU Residual . . . 21
LMR — Load External Data and Memory into Register . . . 23
LRM — Store Register Data into Memory and External Devices . . . 25
LRR — Register-to-Register Transfer (Copy from point B to point A) . . . 27

CMP — Hardware Operand Comparison . . . 29
JMP — Unconditional Program Branch . . . 31
JCR — Conditional Branch on Carry . . . 33
JNZ — Conditional Branch on Non-Zero . . . 35
JZ — Conditional Branch on Zero . . . 37
AND — Bitwise Logical AND . . . 39
OR — Bitwise Logical OR . . . 41
NAND — Bitwise Logical NAND . . . 43
NOR — Bitwise Logical NOR . . . 45
NOT — Bitwise Inversion (One's Complement) . . . 47
XOR — Bitwise Exclusive OR . . . 49
XNOR — Bitwise Equivalence . . . 51
INC — Atomic Increment by 1 . . . 53
DEC — Atomic Decrement by 1 . . . 55
PUSH — Push Word onto Execution Stack . . . 57
POP — Pop Word from Execution Stack . . . 59
BSL — Iterative Hardware Shift Left . . . 61
BSR — Iterative Hardware Shift Right . . . 63
MSP — Manage Special Registers (FREG, PTREG, SREG, VEC) . . . 65
MOD — Integer Modulo Operation . . . 67
INTR — Software Interrupt via 3-Bit Vector Index (1 Byte) . . . 69
RSV29 — Architectural Extension Slot . . . 71
RSV30 — Architectural Extension Slot . . . 73
RSV31 — Architectural Extension Slot . . . 75

7 Physical Package and Pinout (50-Pin Form Factor) 77
7.1 Pin Descriptions . . . 77

8 Interrupt Subsystem and Vector Processing 78
8.1 Unified 16-Vector Interrupt Space . . . 78
8.2 Hardware Handshake Protocol (RDI → IA → D0–D7) . . . 78
8.3 Context Preservation on 24-Bit Stack and Handler Entry . . . 79
8.4 Return from Interrupt . . . 79

9 LIM M2 Assembly Language and Compiler (`limasm.py`) 80
9.1 Register Naming and Operand Size Suffixes . . . 80
9.2 Register Arithmetic, Unary Ops, and Memory Access (LMR, LRM) . . . 80
9.3 Special Register Manipulation (MSP) and Multi-Dereferencing via PTREG . . . 80

10 Conclusion 80

1 Document Scope and Architectural Philosophy

This document serves as the formal architectural reference manual and hardware specification for the **LIM M2** microprocessor. The LIM M2 architecture has been deliberately designed to deliver deterministic execution, robust electrical integrity, and superior clock scaling.

A defining cornerstone of the LIM M2 datapath is the **total elimination of internal tri-state (high-impedance / Hi-Z) buses**. By substituting shared capacitive wires with actively driven multiplexer trees (Point-to-Point MUX Routing), the architecture prevents shoot-through currents, eliminates bus contention risks, and guarantees sharp, symmetrical transition edges.

> **Instruction Code Allocation**
> The LIM M2 instruction set architecture utilizes a contiguous 5-bit primary opcode space: NOP=0, ADD=1, ..., MSP=26. Opcode values 27–31 are reserved for next-generation hardware extensions.

1.1 Notational Conventions

Notation	Architectural Meaning
dst	Destination register targeted for writeback
src, srcA, srcB	Source register operand identifiers
imm	Immediate constant scalar embedded within the instruction
addr	Memory address (16-bit primary or 24-bit extended pointer)
cfg	3-bit operational mode configuration field
ZF, CF, OF	Hardware status flags (Zero, Carry, and Overflow) within FREG

1.2 Unified Instruction Syntax Philosophy

The LIM M2 processor architecture strictly enforces an orthogonal operand convention: **the destination operand (target, "where to") is always declared first, followed by input source operands ("where from")**. This ensures clean, deterministic code structure across all instructions:

- **Arithmetic Operations (3 Operands):** ADD Rd, Rs1, Rs2 denotes Rd $\leftarrow$ Rs1 $+$ Rs2 (e.g., ADD R0, R1, R2 computes $R0 = R1 + R2$); similarly for SUB, MUL, DIV.
- **Logic Operations (2 Operands):** AND Rd, Rs denotes Rd $\leftarrow$ Rd AND Rs (e.g., AND R0, R1 computes $R0 = R0 \ \& \ R1$); similarly for OR, NAND, NOR, XOR, XNOR.
- **Single-Byte Unary Operations (1 Operand):** INC Rd (Rd $\leftarrow$ Rd $+ 1$), DEC Rd (Rd $\leftarrow$ Rd $- 1$), and NOT Rd (Rd $\leftarrow \sim$ Rd). Operate on a single register Rd in place. Encoded in exactly 1 byte ([7:3] opcode, [2:0] Rd), no second byte is fetched from memory.
- **Register-to-Register Transfers:** LRR Rd, Rs executes a direct point-to-point copy from source (point B, Rs) to destination (point A, Rd): Rd $\leftarrow$ Rs (e.g., LRR R0, R1 computes $R0 = R1$).
- **Memory Loads (LMR):** the primary register argument Rd designates the processor register receiving loaded data.

2 Processor Overview

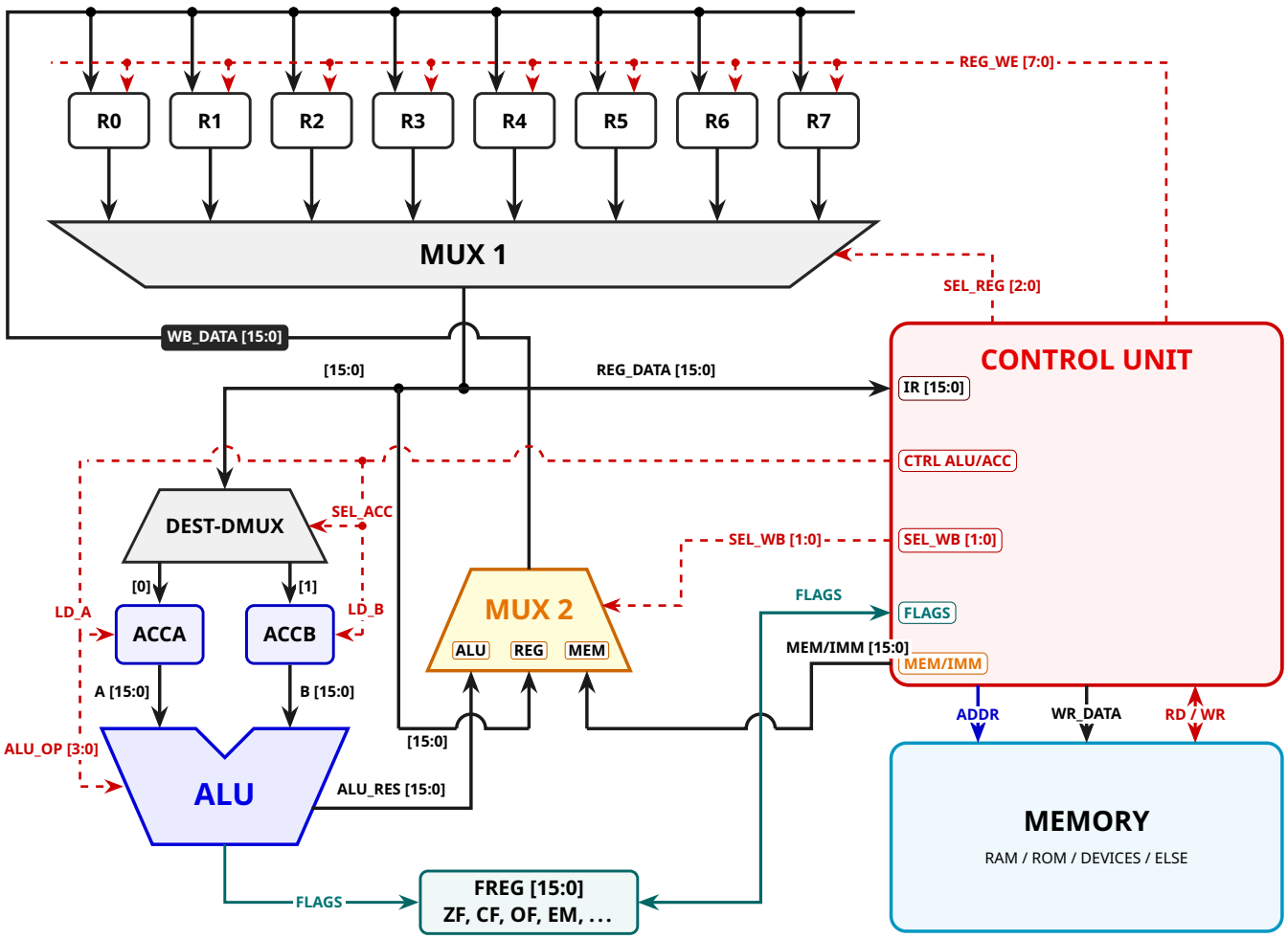
The **LIM M2** is a versatile, high-throughput 16-bit microprogrammed processor engineered for integer-intensive computation. Architectural highlights include:

- **Conflict-Free Multiplexer Datapath:** Internal signals are steered purely by push-pull multiplexer

trees (**MUX 1**, **DEST-DMUX**, and **MUX 2**). This modern design is perfectly suited for high-density ASIC and modern FPGA synthesis.

- **Dual Accumulator Staging:** Dedicated pre-ALU accumulators (ACCA and ACCB) isolate the arithmetic core from the general register file, minimizing propagation path skew.
- **Hardware ALU Auto-Chaining (Multi-Word Arithmetic):** The ALU integrates an internal residual register (ALU_RESIDUAL). When an addition overflows 16 bits ($> 2^{16} - 1$), a subtraction produces a borrow (< 0), or multiplication/division produces high-word residuals, the excess is retained in hardware and automatically accumulated or subtracted in the next mathematical operation. This enables continuous multi-word arithmetic (32, 48, 64+ bits) without dedicated ADC/SBB instructions. The chain is flushed using NOP.
- **Direct Reg-to-Reg Bypass:** A low-latency path from MUX 1 directly into MUX 2 enables single-cycle register transfers without locking ALU resources.
- **Native 32-Bit Linear Addressing (Up to 4 GB):** A native 32-bit address bus (A0–A31) and dedicated 32-bit pointer registers (PTREG with auto-increment and SREG stack pointer), accompanied by 32 system vector registers (`$ir0..$ir15`, `$pv0..$pv15`), provide direct linear access across the entire 4 Gigabyte physical address space without bus multiplexing, paging windows, or bank switching.
- **High Clock Scaling Potential:** Unburdened by capacitive bus decay, the LIM M2 operates comfortably at a baseline frequency of **200 MHz**, scaling toward **700 MHz and above** in optimized silicon.

2.1 Conceptual Datapath Schematic



3 Architecturally Visible Registers

The programmer-visible register set of the LIM M2 processor comprises a clean, orthogonal hierarchy:

Register	Width	Architectural Purpose and Features
R0–R7	16-bit	General-Purpose Registers (GPR). Low-latency reading via MUX 1; atomic writeback from the WB_DATA highway under per-register write gating.
SREG	24-bit	Dedicated Stack Register. Supports efficient hardware frame and subroutine stack management with linear 24-bit stack addressing (up to 16 MB).
PTREG	24-bit	High-Precision Pointer Register. Provides flat linear addressing across 16 MB with optional hardware auto-increment.
FREG	16-bit	Status and Control Register. Unifies mathematical ALU condition flags with processor operational control bits.

3.1 Flag and Status Register (FREG) Mapping

The 16-bit FREG provides granular status monitoring and execution mode management:

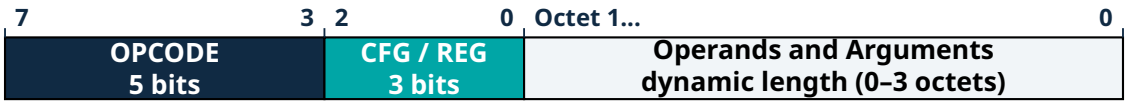
Bit	Mnemonic	Functional Description
0	ZF	Zero Flag: Asserted when the execution result equals zero.
1	CF	Carry Flag: Captures arithmetic carry/borrow out of the most significant bit.
2	OF	Overflow Flag: Two's complement signed overflow indicator.
3	EM	Error Math: Hardware trap indicator for mathematical faults (e.g., divide-by-zero).
4–5	CMP	Compare Result: Hardware comparator status (00: reset/none/error, 01: less, 10: equal, 11: greater).
6–7	CLK	Clock Mode: Frequency prescaler and core clock domain configuration.
8	PIE	Program Interrupt Enable: Master mask for software interrupts.
9	MIE	Master Interrupt Enable: Hardware external interrupt enable.
10	DWR	Disable Wait for Ready: Forces immediate bus cycles ignoring slave wait states.
11	PRV	Privilege Mode: 0 = User Mode, 1 = Supervisor Mode (resets to 1; protected by 8 Privilege Zones $pv0..$pv15).
12	AIP	Auto-Increment PTREG: Automatic hardware increment of PTREG upon memory access.
13	DBIT	Displaced Bit: Dedicated buffer for the bit ejected during shift operations.
14	SMOD	Shift Mode: Cyclic shift mode selector (0: zero-fill, 1: rotate through DBIT).
15	INTR	In Interrupt: Active interrupt execution flag (when 1, subsequent interrupts are locked out).

4 Machine Instruction Format

LIM M2 instructions follow an orthogonal, modular encoding scheme with deterministic hardware decoding. The 5-bit primary opcode resides in the upper 5 bits of the initial byte (Octet 0), enabling single-cycle microcode ROM dispatch without instruction pre-decoding pipeline bubbles.

4.1 Universal Instruction Word Format

Every machine instruction begins with a fixed-size header, followed by dynamic operands and immediate extensions:



The lower 3 bits of the initial octet (`[2:0]`) serve a dual, deterministic architectural role:

- In register operations (arithmetic, unary, logic, shift, register move, stack), bits $[2:0]$ directly encode the **first register operand** (Rd or `Rs1`), enabling full 8-register addressing (`R0`–`R7`) without unnecessary decoding overhead.
- In instructions requiring hardware mode selection or memory/special-register selectors (`LMR`, `LRM`, `MSP`, `JMP`, `JCR`, `JNZ`, `JZ`), bits $[2:0]$ serve as the configuration/mode selector (CFG / MOD / SRC / DST).
- In zero-operand control instructions (`NOP`, `HALT`, `CLI`, `STI`, `RET`), these bits are set to `000`.

4.2 Class I: Single-Byte Instructions (1 Byte, 0 Extra Octets)

Control commands (`NOP`, `HALT`, `CLI`, `STI`, `RET`) and unary register operations (`NOT`, `INC`, `DEC`) execute from a single fetched byte without requesting a second byte from memory:

- Argument-less commands have CFG set to `000`.
- Unary instructions (`INC Rd`, `DEC Rd`, `NOT Rd`) encode the target register index Rd (`R0`–`R7`) directly in bits $[2:0]$.



Octet 0: Fixed 1-Byte Instruction

4.3 Class II: Three-Operand Arithmetic Instructions (2 Bytes)

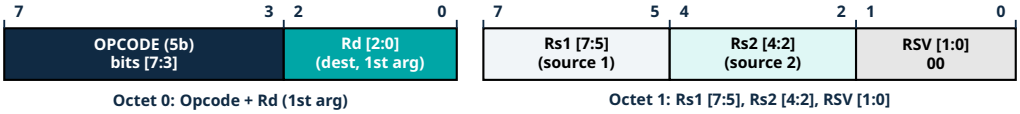
Arithmetic instructions (`ADD`, `SUB`, `MUL`, `DIV`, `MOD`) adhere to the universal syntax paradigm: **destination first, then operands**:

$$\texttt{ADD Rd, Rs1, Rs2} \implies \texttt{Rd} \leftarrow \texttt{Rs1} + \texttt{Rs2} + \texttt{ALU_RESIDUAL}$$

$$\texttt{MOD Rd, Rs1, Rs2} \implies \texttt{Rd} \leftarrow \texttt{Rs1} \pmod{\texttt{Rs2}}$$

The first instruction argument (destination register Rd) is encoded directly in the lower 3 bits of Octet 0, allowing full 3-bit addressing (`R0`–`R7`) for all three operands:

- **Octet 0:** Bits $[7:3]$ — Opcode (5 bits); Bits $[2:0]$ — Destination register Rd (3 bits, `R0`–`R7`, 1st argument).
- **Octet 1:** Bits $[7:5]$ — First source operand `Rs1` (3 bits, `R0`–`R7`); Bits $[4:2]$ — Second source operand `Rs2` (3 bits, `R0`–`R7`); Bits $[1:0]$ — Reserved (`00`).



4.4 Class III: Two-Operand Logic Instructions (2 Bytes)

Logic instructions (AND, OR, NAND, NOR, XOR, XNOR) take two operand registers:

$$\texttt{AND Rd, Rs} \quad \Longrightarrow \quad \text{Rd} \leftarrow \text{Rd AND Rs}$$

- **Octet 0:** Bits $[7:3]$ — Opcode; Bits $[2:0]$ — Destination register Rd (R0–R7, 1st argument).
- **Octet 1:** Bits $[7:5]$ — Second register operand Rs (R0–R7); Bits $[4:0]$ — Reserved (00000_2).



4.5 Class IV: Register-to-Register Move (LRR, 2 Bytes)

Instruction `LRR Rd, Rs` copies from source (point B, Rs) to destination (point A, Rd):

$$\texttt{LRR Rd, Rs} \quad \Longrightarrow \quad \text{Rd} \leftarrow \text{Rs}$$

Data moves via the dedicated bypass path **MUX 1** $\rightarrow$ **MUX 2** bypassing the ALU in 1 cycle without affecting flags. Encoding: Octet 0 contains opcode `00111` and destination register Rd (`[2:0]`). Octet 1 contains source register Rs (`[7:5]`) and reserved bits (`[4:0]` = `00000`).

4.6 Class V: Hardware Comparator (CMP, 2 Bytes)

Instruction `CMP Rs1, Rs2` non-destructively compares two register values:

- **Octet 0:** Opcode `01000`, bits $[2:0]$ specify the first source operand Rs1 (1st argument).
- **Octet 1:** Bits $[7:5]$ specify the second source operand Rs2; bits $[4:0]$ are reserved (`00000`).

The comparison updates the 2-bit field FREG.CMP (bits $[5:4]$):

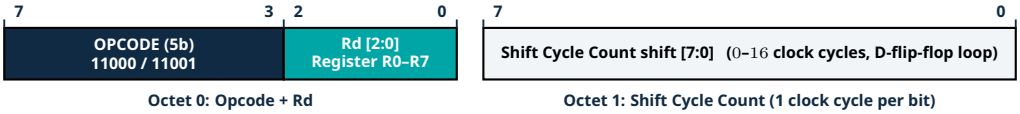
Value of FREG.CMP	Condition	Meaning
`01` (01_2)	Rs1 $<$ Rs2	First operand is strictly less than second
`10` (10_2)	Rs1 $=$ Rs2	Operands are equal
`11` (11_2)	Rs1 $>$ Rs2	First operand is strictly greater than second
`00` (00_2)	Reset / Error	Comparator inactive or reset state

4.7 Class VI: Iterative D-Flip-Flop Hardware Shift (BSL, BSR)

Shift instructions `BSL Rd, shift` and `BSR Rd, shift` are physically implemented as an **iterative 16-stage D-flip-flop shift register** costing ≈ 300 transistors instead of $\approx 30,000$ for a barrel matrix shifter:

- Shifting executes cycle-by-cycle (hardware `for` loop): a 16-bit shift takes exactly 15 cycles longer than a 1-bit shift (1 cycle per bit shifted).
- Octet 0: Upper 5 bits hold opcode (BSL: `11000`, BSR: `11001`), lower 3 bits (`[2:0]`) select register Rd.

- Octet 1: Entire byte (`[7:0]`) specifies the shift count (0–16 cycles).
- Injected bits are controlled by FREG: SMOD $= 1$ rotates ejected bits into the opposite side; SMOD $= 0$ injects `FREG.DBIT`.



4.8 Class VII: Stack Operations (PUSH, POP, 2 Bytes)

Stack instructions push/pop registers via the 32-bit stack pointer SREG:

- **Octet 0:** Opcode (PUSH: `10110`, POP: `10111`), bits $[2:0]$ directly designate operand register (Rs for push, Rd for pop, 1st argument).
- **Octet 1:** Bits $[7:0]$ are reserved (00000000_2).

4.9 Class VIII: Special Register Manipulation (MSP, Opcode 26)

The MSP instruction (Opcode `11010`, 26_{10}) provides a modular interface to configure all system, address, and protection facilities: FREG, PTREG, SREG, and the 32 vector registers for interrupt service routines and Privilege Zones (`$ir0..$ir15`, `$pv0..$pv15`). Instruction length spans from 2 to 6 bytes depending on the addressing mode.

4.9.1 Format 8.1: FREG Load (MOD = 0, REG = 00)

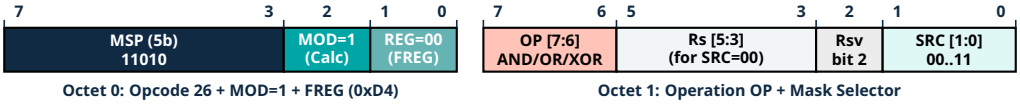
The `MSP.ld $freg, <src>` command directly initializes flags.



- SRC $= 00_2$ (2 bytes): Load from GPR register Rs (`MSP.ld $freg, $rs`).
- SRC $= 01_2$ (4 bytes): Immediate constant `imm16` in octets 2–3 (`MSP.ld $freg, 0x0000`).
- SRC $= 10_2$ (2 bytes): Read from memory at pointer `[$ptreg]` (`MSP.ld $freg, [$ptreg]`).
- SRC $= 11_2$ (4 bytes): Read from memory at direct 16-bit address `[addr16]` (`MSP.ld $freg, [0x1000]`).

4.9.2 Format 8.2: FREG Bitwise Logic (MOD = 1, REG = 00)

The `MSP.and`, `MSP.or`, `MSP.xor`, `MSP.xnor` instructions apply bitwise masks to flags.

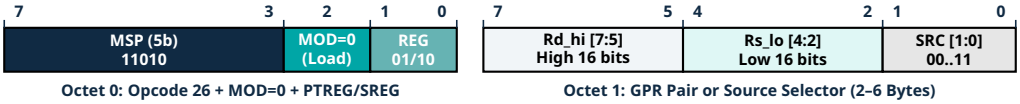


- **Operation code OP**$[7:6]$**:** 00_2 = AND, 01_2 = OR, 10_2 = XOR, 11_2 = XNOR.
- **Hardware Supervisor Mode Protection (PRV):** The core automatically boots and resets in Supervisor Mode (PRV=1). The early startup code (bootloader/kernel) initializes interrupt vectors (`$ir0..$ir15`) and Privilege Zone boundaries (`$pv0..$pv15`), then drops privileges to User Mode (PRV=0). While in User Mode, switching to PRV=1 or reconfiguring zone registers is hardware-permitted strictly when the current PC lies within one of the 8 registered Privilege

Zones ($\exists k \in [0..7] : \$pv_{2k} \leq \text{PC} \leq \$pv_{2k+1}$). **Crucial Architectural Rule:** Exiting a Privilege Zone does **not** automatically drop PRV=1 — the OS/program is strictly responsible for clearing PRV=0 before returning control to unprivileged code.

4.9.3 Format 8.3: PTREG & SREG Pointer Load (MOD = 0, REG = 01 / 10)

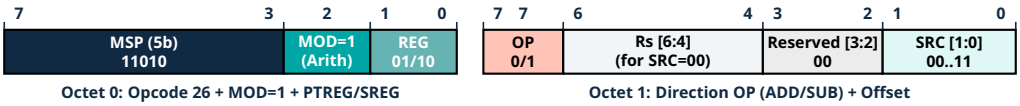
Commands `MSP.ld $ptreg` (REG $= 01_2$, opcode `0xD1`) and `MSP.ld $sreg` (REG $= 10_2$, opcode `0xD2`) initialize 32-bit linear address pointers.



- SRC $= 00_2$ (2 bytes): Assemble 32-bit address from GPR pair Rd_hi:Rs_lo (`MSP.ld $ptreg, $0, $1`).
- SRC $= 01_2$ (4–6 bytes): Immediate address `imm32` (6 bytes) or `imm16` (4 bytes).
- SRC $= 10_2$ (**2 bytes**): **Native single-cycle pointer dereferencing** `[$ptreg]` (`PTREG <- MEM[PTREG]`).
- SRC $= 11_2$ (4–6 bytes): Load pointer from memory at absolute address `[addr]` (`MSP.ld $ptreg, [0xA000]`).

4.9.4 Format 8.4: PTREG & SREG Pointer Arithmetic (MOD = 1, REG = 01 / 10)

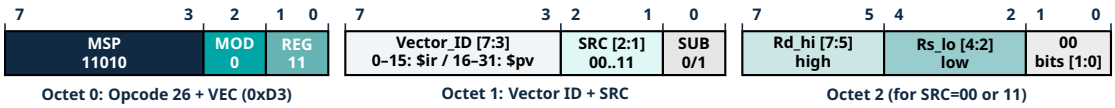
Commands `MSP.add` (OP $= 0$) and `MSP.sub` (OP $= 1$) execute rapid hardware offsetting of buffer and stack pointers.



- OP $= 0$: ADD (+offset to pointer), OP $= 1$: SUB (-offset from pointer).
- Offset source: GPR register SRC $= 00_2$ (2 bytes), immediate constant SRC $= 01_2$ (4 bytes), read offset via SRC $= 10_2$ (`[$ptreg]`, 2 bytes) or SRC $= 11_2$ (`[addr16]`, 4 bytes).

4.9.5 Format 8.5: Vector Registers Load (MOD = 0, REG = 11)

Command `MSP.ld` loads 32-bit addresses into one of 32 vector registers: `$ir0`..`$ir15` (interrupt vectors, IDs 0–15) and `$pv0`..`$pv15` (Privilege Zone boundaries, IDs 16–31).



- **Variant 5.1 (SRC $= 00_2$, SUB $= 0$, 3 bytes):** Load vector from GPR pair Rd_hi:Rs_lo (`MSP.ld $ir0, $0, $1`).
- **Variant 5.2 (SRC $= 01_2$, SUB $= 0$, 6 bytes):** Direct 32-bit address vector `imm32` in octets 2–5 (`MSP.ld $pv0, 0x00008000`).
- **Variant 5.3 (SRC $= 10_2$, SUB $= 0$, 2 bytes):** Read vector from memory at pointer `[$ptreg]` (`MSP.ld $ir1, [$ptreg]`).
- **Variant 5.4 (SRC $= 11_2$, SUB $= 0$, 3 bytes):** Read vector from memory via 32-bit pointer in GPR pair `[$rd:$rs]` (`MSP.ld $ir2, [$0:$1]`).
- **Variant 5.5 (SRC $= 11_2$, SUB $= 1$, 6 bytes):** Read vector from memory at direct 32-bit address `[addr32]` (`MSP.ld $ir0, [0x00005000]`).

4.10 Class IX: Branch Instructions (2–3 Bytes)

Branches (JMP, JCR, JNZ, JZ) support 3 modes selected via CFG (bits $[2:0]$ of octet 0):

- **MOD = 0 (000): Immediate 16-bit address (3 bytes).** Followed by octets 1 and 2 encoding target address in the 64-KB code space.
- **MOD = 1 (001): Indirect register branch (2 bytes).** Octet 1 bits $[7:5]$ designate source register Rs.
- **MOD = 2 (010): 24-bit pointer branch via PTREG.** Jump to flat 24-bit physical address in PTREG.

4.11 Class X: Memory Transfer Instructions (LMR, LRM, 2–4 Bytes)

Load (LMR) and Store (LRM) encode flexible operand sizing and addressing in CFG:

- Bit $[2]$: Operand size SZ ($0 = 8$-bit byte, $1 = 16$-bit word).
- Bits $[1:0]$: Addressing mode SRC / DST:
  - **0: Inline Immediate Data:** 2 bytes (SZ $= 0$) or 3 bytes (SZ $= 1$).
  - **1: Immediate Pointer:** 4 bytes. Octet 1 designates register, octets 2–3 hold 16-bit address.
  - **2: 24-Bit PTREG Pointer:** 2 bytes. Automatically auto-increments if FREG.AIP is set.
  - **3: Register Pair 24-bit Address:** 2 bytes. Octet 1 encodes $[7:5]$ (Arg1 / upper address) and $[4:2]$ (Arg2 / lower 16-bit address). In LMR, Arg1 receives loaded data.

5 Instruction Set Architecture Summary

No.	Code	Mnemonic	Class	Functional Summary
0	00000	NOP	Control	No operation and ALU residual chain flush
1	00001	ADD	Arithmetic	Addition with automatic multi-word carry chaining
2	00010	SUB	Arithmetic	Subtraction with automatic multi-word borrow chaining
3	00011	MUL	Arithmetic	Integer multiplication with upper word residual latching
4	00100	DIV	Arithmetic	Integer division with remainder latched in ALU residual
5	00101	LMR	Memory	Load byte/word into register (src 0–3, size 8/16-bit)
6	00110	LRM	Memory	Store register byte/word into memory (dst 0–3, size 8/16-bit)
7	00111	LRR	Transfer	Low-latency register-to-register copy via bypass MUX
8	01000	CMP	Compare	Arithmetic compare without modifying operand registers
9	01001	JMP	Branch	Unconditional branch (direct, indirect, 24-bit pointer)
10	01010	JCR	Branch	Conditional branch on Carry Flag (CF)
11	01011	JNZ	Branch	Conditional branch on Non-Zero (ZF = 0)
12	01100	JZ	Branch	Conditional branch on Zero (ZF = 1)
13	01101	AND	Logic	Bitwise logical AND
14	01110	OR	Logic	Bitwise logical OR
15	01111	NAND	Logic	Bitwise logical NAND
16	10000	NOR	Logic	Bitwise logical NOR
17	10001	NOT	Logic	Bitwise bit inversion (one's complement)
18	10010	XOR	Logic	Bitwise exclusive OR
19	10011	XNOR	Logic	Bitwise equivalence
20	10100	INC	Arithmetic	Atomic increment operand by 1

No.	Code	Mnemonic	Class	Functional Summary
21	10101	DEC	Arithmetic	Atomic decrement operand by 1
22	10110	PUSH	Stack	Push word onto execution stack
23	10111	POP	Stack	Pop word from execution stack into destination
24	11000	BSL	Shift	Barrel shift left with configurable fill
25	11001	BSR	Shift	Barrel shift right with discarded bit capture
26	11010	MSP	System	Manage special registers (FREG, PTREG, SREG)
27	11011	MOD	Arithmetic	Integer remainder operation (modulo)
28	11100	INTR	System	Software interrupt via 3-bit vector index (bits [2:0])
29	11101	RSV29	Reserved	Reserved for floating-point coprocessor
30	11110	RSV30	Reserved	Reserved for hardware cryptographic acceleration
31	11111	RSV31	Reserved	Reserved for future architectural expansion

6 Detailed Instruction Specifications

The following pages detail the formal execution profiles for all 32 opcode allocations.

NOP

Hardware No-Operation and Pipeline Flush

00000
Control

Mnemonic	NOP
Opcode	`00000` (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Control
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Hardware No-Operation and Pipeline Flush.

Architectural Semantics

```
Syntax:  NOP
Operation:  PC <- PC + 1, ALU_RESIDUAL <- 0, FREG.CF <- 0, FREG.OF <- 0
Example:  NOP ; Flush ALU carry chain, align cycles
```

Configuration and Operands



Octet 0 (1 byte, 0 extra bytes)

Fixed 1-byte instruction length. No operands required; cfg field set to `000` (reserved for delay calibration). No second byte fetched from memory.

Flags Affected (FREG)

Clears carry (CF), overflow (OF), and borrow tracking flags; preserves other flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Performs pipeline cycle alignment and flushes the internal ALU residual accumulator, terminating any chained multi-word carry/borrow sequence before starting new independent calculations.

TBD markers denote configurable hardware options.

ADD

Addition with Automatic Multi-Word Carry Chaining

00001
Arithmetic

Mnemonic	ADD
Opcode	00001 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

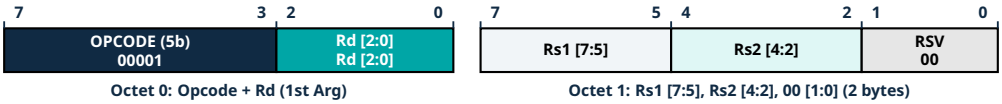
Instruction Purpose

Addition with Automatic Multi-Word Carry Chaining.

Architectural Semantics

```
Syntax:  ADD arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 + arg2
Operation:  sum <- arg1 + arg2 + ALU_RESIDUAL
arg0 <- sum[15:0], ALU_RESIDUAL <- (sum > 0xFFFF ? 1 :  0)
Example:  ADD R0, R1, R2 ; R0 = R1 + R2 (registers R0-R7)
```

Configuration and Operands



Octet 0: Opcode 00001, bits $[2:0]$ select destination register Rd (R0–R7, 1st argument).
Octet 1: Bits $[7:5]$ select first addend Rs1 (R0–R7); bits $[4:2]$ select second addend Rs2 (R0–R7); bits $[1:0]$ are reserved (00). Instruction length: exactly 2 bytes.

Flags Affected (FREG)

Sets carry flag CF and overflow OF if sum exceeds $2^{16} - 1$; updates ZF based on dst.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Delivers seamless multi-word arithmetic (32, 48, 64+ bits). If the addition exceeds 16 bits ($> 2^{16} - 1$), the carry (+1) is latched into the internal ALU residual register and automatically added during the subsequent ADD operation without requiring distinct ADC opcodes. The lower 16 bits are deterministically stored in dst. The chain is reset via NOP or overwritten by a new operation.

SUB

Subtraction with Automatic Multi-Word Borrow Chaining

00010
Arithmetic

Mnemonic	SUB
Opcode	00010 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

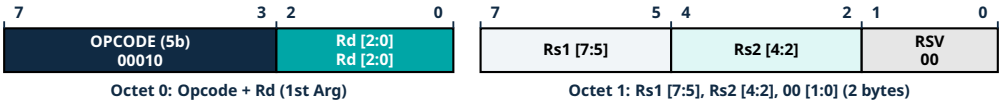
Instruction Purpose

Subtraction with Automatic Multi-Word Borrow Chaining.

Architectural Semantics

```
Syntax:  SUB arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 - arg2
Operation:  diff <- arg1 - arg2 - ALU_RESIDUAL
arg0 <- diff[15:0], ALU_RESIDUAL <- (diff < 0 ?  1 :  0)
Example:  SUB R0, R1, R2 ; R0 = R1 - R2 (registers R0-R7)
```

Configuration and Operands



Octet 0: Opcode 00010, bits $[2:0]$ select destination register Rd (R0–R7, 1st argument).
Octet 1: Bits $[7:5]$ select minuend Rs1 (R0–R7); bits $[4:2]$ select subtrahend Rs2 (R0–R7); bits $[1:0]$ are reserved (00). Instruction length: exactly 2 bytes.

Flags Affected (FREG)

Sets borrow flag (CF) and OF on negative result (< 0); updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Enables continuous multi-word subtraction. When a borrow occurs (< 0), the borrow (-1) is retained in the internal ALU register and automatically subtracted during the next SUB operation. The lower 16-bit word is deterministically preserved in dst. Inserting a NOP breaks the chain for independent subtraction.

TBD markers denote configurable hardware options.

MUL

Multiplication with Upper Word Residual Retention

00011
Arithmetic

Mnemonic	MUL
Opcode	00011 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

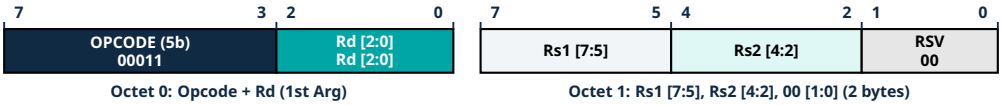
Instruction Purpose

Multiplication with Upper Word Residual Retention.

Architectural Semantics

```
Syntax:  MUL arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 * arg2
Operation:  prod <- arg1 * arg2
arg0 <- prod[15:0], ALU_RESIDUAL <- prod[31:16]
Example:  MUL R0, R1, R2 ; R0 = R1 * R2 (upper 16 bits in ALU_RESIDUAL)
```

Configuration and Operands



Octet 0: Opcode 00011, bits $[2:0]$ select destination register Rd (R0–R7, 1st argument).
Octet 1: Bits $[7:5]$ select first factor Rs1 (R0–R7); bits $[4:2]$ select second factor Rs2 (R0–R7); bits $[1:0]$ are reserved (00). Upper 16 bits are automatically retained in ALU_RESIDUAL. Instruction length: 2 bytes.

Flags Affected (FREG)

Updates ZF; sets OF/CF if the upper 16 bits of the product (prod[31:16]) are non-zero.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.

4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

For 32-bit products exceeding the 16-bit register width, the low 16 bits are placed into dst, while the upper 16 bits are retained in the internal ALU residual register. In a subsequent addition, this residual is automatically accumulated (hardware MAC support / bigint multiplication). The upper word can also be extracted by adding the register to itself after clearing. Executing NOP flushes the residual register.

TBD markers denote configurable hardware options.

DIV

Division with Remainder Latched in ALU Residual

00100
Arithmetic

Mnemonic	DIV
Opcode	00100 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

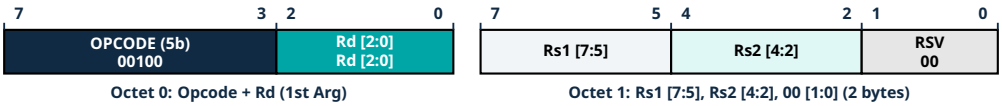
Instruction Purpose

Division with Remainder Latched in ALU Residual.

Architectural Semantics

```
Syntax:  DIV arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 / arg2
Operation:  arg0 <- arg1 / arg2
ALU_RESIDUAL <- arg1 % arg2
Example:  DIV R0, R1, R2 ; R0 = R1 / R2 (remainder modulo in ALU_RESIDUAL)
```

Configuration and Operands



Octet 0: Opcode 00100, bits $[2:0]$ select destination register Rd (R0–R7, 1st argument).
Octet 1: Bits $[7:5]$ select dividend Rs1 (R0–R7); bits $[4:2]$ select divisor Rs2 (R0–R7); bits $[1:0]$ are reserved (00). Remainder (modulo) is automatically latched in ALU_RESIDUAL. Instruction length: 2 bytes.

Flags Affected (FREG)

Sets EM (Error Math) flag on division by zero without core stall; updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.

4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Stores the integer quotient in dst while preserving the remainder (modulo) in the internal ALU residual register for immediate reuse in subsequent operations or clearing via NOP.

TBD markers denote configurable hardware options.

LMR

Load External Data and Memory into Register

00101
Memory

Mnemonic	LMR
Opcode	00101 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Memory
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Load External Data and Memory into Register.

Architectural Semantics

```
Syntax:  LMR arg0(REG), <SRC_MODE> :  arg0 = MEM[...]
Modes:  SRC=0:  LMR R0, [PC+] ; Imm Data (1-2 bytes)
SRC=1:  LMR R0, [[PC+]] ; Imm Pointer
SRC=2:  LMR R0, [PTREG] ; Pointer reg (AIP auto-inc)
SRC=3:  LMR R0(R_hi), R1(R_lo) ; 24-bit composite address
Operation:  arg0 <- MEM[EA] (upper address bits in R0 overwritten)
Example:  LMR R0, R1 ; EA=(R0«16)|R1, load MEM[EA] into R0
```

Configuration and Operands



Octet 0: bit $[8]$ (SZ) specifies operand width ($0 = 1$ byte, $1 = 2$ bytes / word); bits $[10:9]$ (SRC) select the data source (0 — immediate literal following instruction, 1 — immediate pointer following instruction, 2 — indirect via 24-bit pointer register PTREG, 3 — indirect via register pair).
Octet 1: bits $[7:5]$ select register `Arg1` (`R_hi` / destination), bits $[4:2]$ select register `Arg2` (`R_lo`). Under SRC $= 3$, the user specifies two 16-bit registers forming a 32-bit number, of which 24 bits are used in practice for physical addressing: `Arg1` holds upper address bits and `Arg2` holds lower 16 address bits. Notably, `Arg1` also acts as the destination register: upon memory retrieval, the upper address bits in `Arg1` are overwritten with the loaded data.

Flags Affected (FREG)

All flags in status register FREG are preserved without alteration.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

The LMR instruction is the fundamental ingestion pathway bringing external data and memory into the LIM M2 general-purpose register file ($R0$–$R7$). The architecture unifies four orthogonal modes with native 24-bit addressing:
1. Under SRC $= 0$, immediate literals (1 or 2 bytes) are fetched directly from the instruction stream with automated PC advancement.
2. Under SRC $= 1$, an immediate pointer is read from code space, followed by data dereferencing.
3. Under SRC $= 2$, memory is addressed via the dedicated 24-bit pointer register PTREG. If the auto-increment flag FREG.AIP (bit 12) is active, PTREG automatically advances by $+1$ or $+2$ in hardware, enabling single-cycle streaming array ingestion without software loop overhead.
4. Under SRC $= 3$, the user selects two 16-bit registers (Arg1 and Arg2) forming a 32-bit value, of which 24 bits are used for memory addressing. The user specifies that the upper address bits are placed in Arg1 and the lower 16 bits in Arg2. In this mode, register Arg1 (which held the upper address bits) also serves as the destination register: the retrieved data is saved into Arg1, overwriting its previous high-address value, while Arg2 continues to retain the lower 16 address bits.

TBD markers denote configurable hardware options.

LRM

Store Register Data into Memory and External Devices

00110
Memory

Mnemonic	LRM
Opcode	00110 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Memory
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Store Register Data into Memory and External Devices.

Architectural Semantics

```
Syntax:  LRM arg0(REG), <DST_MODE> :  MEM[...]  = arg0
Modes:  DST=0:  LRM R0, [PC+] ; Inline code memory
DST=1:  LRM R0, [[PC+]] ; Imm Pointer
DST=2:  LRM R0, [PTREG] ; Pointer reg (AIP auto-inc)
DST=3:  LRM R0(R_hi), R1(R_lo) ; 24-bit composite address
Operation:  MEM[EA] <- arg0[15:0] (or low byte if SZ=0)
Example:  LRM R0, R1 ; EA=(R0«16)|R1, store R0 into MEM[EA]
```

Configuration and Operands



Octet 0: bit $[8]$ (SZ) specifies operand width ($0 = 1$ byte, $1 = 2$ bytes / word); bits $[10:9]$ (DST) select the destination target (0 — inline memory directly following instruction, 1 — immediate pointer following instruction, 2 — indirect via 24-bit pointer register PTREG, 3 — indirect via register pair). **Octet 1:** bits $[7:5]$ select source register Arg1 (Rs / R_hi), bits $[4:2]$ select register Arg2 (R_lo). Under DST $= 3$, two registers form a 32-bit composite word of which 24 bits are used for physical addressing (Arg1 holds upper address bits and source data, Arg2 holds lower 16 bits). Overwriting existing data at the destination is the programmer's responsibility.

Flags Affected (FREG)

All flags in status register FREG are preserved without alteration.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

The LRM instruction is the fundamental egress pathway transferring register data from the LIM M2 general-purpose register file ($R0$–$R7$) to memory and memory-mapped I/O peripherals. The architecture unifies four orthogonal addressing modes:
1. Under DST $= 0$, the processor stores 1 or 2 bytes directly into memory following the instruction (PC) with automated PC advancement. This mode enables self-modifying routines and inline buffer writes; data destruction at the target address is the programmer's responsibility.
2. Under DST $= 1$, an immediate 16-bit pointer is read from code space, and data is stored to that dereferenced address.
3. Under DST $= 2$, memory is addressed via the dedicated 24-bit pointer register PTREG. If the auto-increment flag FREG.AIP (bit 12) is active, PTREG automatically advances by $+1$ or $+2$ in hardware, enabling single-cycle streaming array transfers without loop overhead.
4. Under DST $= 3$, two 16-bit registers are paired: Arg1 and Arg2 form a 32-bit composite integer, whose lower 24 bits specify the physical address (Addr$[23:0]$). Data from Arg1 is stored to the synthesized address, granting direct write access across the full 16 MB address space.

TBD markers denote configurable hardware options.

LRR

Register-to-Register Transfer (Copy from point B to point A)

00111
Transfer

Mnemonic	LRR
Opcode	00111 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Transfer
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Register-to-Register Transfer (Copy from point B to point A).

Architectural Semantics

```
Syntax:  LRR arg0(REG), arg1(REG) : arg0 = arg1
Operation:  arg0 <- arg1 (direct copy from source to destination)
Example:  LRR R0, R1 ; R0 = R1 (value of R1 copied to R0)
```

Configuration and Operands



Octet 0: Opcode + Rd

Octet 1: Rs [7:5] (Length: 2 bytes)

Octet 0: Opcode 00111_2 (0x07), bits $[2:0]$ select destination register Rd (R0–R7, point A, 1st argument).
Octet 1: Bits $[7:5]$ select source register Rs (R0–R7, point B); bits $[4:0]$ are reserved (00000_2). When executed, Rs is copied to Rd in 1 microcycle. Instruction length: 2 bytes.

Flags Affected (FREG)

Preserves all status flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.

4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

The LRR instruction provides the foundational register-to-register move mechanism across general-purpose registers ($R0$–$R7$). It executes a direct transfer from source (point B) to destination (point A) via the dedicated hardware **MUX 1** $\rightarrow$ **MUX 2** bypass datapath without stalling the ALU and without affecting FREG flags.

TBD markers denote configurable hardware options.

CMP

Hardware Operand Comparison

01000
Compare

Mnemonic	CMP
Opcode	01000 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Compare
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

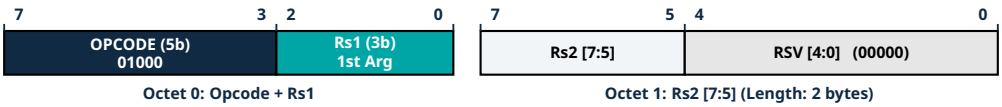
Instruction Purpose

Hardware Operand Comparison.

Architectural Semantics

```
Syntax:  CMP arg0(REG), arg1(REG) : compare(arg0, arg1)
Operation:  if arg0 < arg1 then FREG.CMP <- 01b (less)
if arg0 == arg1 then FREG.CMP <- 10b (equal)
if arg0 > arg1 then FREG.CMP <- 11b (greater)
(state 00b – initial reset / no comparison / error)
Example:  CMP R0, R1 ; 01:  R0<R1, 10:  R0==R1, 11:  R0>R1 (00:  reset/error)
```

Configuration and Operands



Octet 0: Opcode 01000, bits $[2:0]$ specify first operand Rs1 (1st argument).
Octet 1: Bits $[7:5]$ specify second operand Rs2; bits $[4:0]$ are reserved (00000_2).
The result updates the FREG.CMP field (bits $[5:4]$): 01 (Rs1 $<$ Rs2), 10 (Rs1 $=$ Rs2), 11 (Rs1 $>$ Rs2), 00 (reset/error). Register contents are unchanged. Instruction length: 2 bytes.

Flags Affected (FREG)

Updates 2-bit FREG.CMP field (bits [5:4]) as well as ZF, CF, OF flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.

2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Enables swift condition evaluation without data corruption.

TBD markers denote configurable hardware options.

JMP

Unconditional Program Branch

01001
Branch

Mnemonic	JMP
Opcode	01001 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Branch
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Unconditional Program Branch.

Architectural Semantics

```
Syntax:  JMP target(IMM|REG) : PC = target
Operation:  PC <- target
Example:  JMP R0 ; Unconditional jump to address in R0
```

Configuration and Operands



Octet 0: Opcode 01001, bits $[2:0]$ (MOD): 000 — immediate 16-bit address (length 3 bytes); 001 — indirect register branch (length 2 bytes, octet 1 bits $[7:5]$ specify Rs); 010 — 24-bit branch via pointer register PTREG.

Flags Affected (FREG)

Preserves all status flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Atomic reload of the Program Counter in minimal clock cycles.

TBD markers denote configurable hardware options.

JCR

Conditional Branch on Carry

01010
Branch

Mnemonic	JCR
Opcode	01010 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Branch
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Conditional Branch on Carry.

Architectural Semantics

```
Syntax:  JCR target(IMM|REG) : if FREG.CF == 1 then PC = target
Operation:  if FREG.CF == 1 then PC <- target
Example:  JCR R2 ; Jump to address R2 if Carry Flag is set
```

Configuration and Operands



Octet 0: Opcode 01010, bits $[2:0]$ (MOD): 000 (16-bit Imm address, 3 bytes), 001 (indirect via Rs, 2 bytes), 010 (24-bit via PTREG). Jump takes place strictly if Carry Flag FREG.CF = 1.

Flags Affected (FREG)

Flags read without modification.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Foundational primitive for multi-word extended precision arithmetic loops.

JNZ

Conditional Branch on Non-Zero

01011
Branch

Mnemonic	JNZ
Opcode	01011 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Branch
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Conditional Branch on Non-Zero.

Architectural Semantics

```
Syntax:  JNZ target(IMM|REG) : if FREG.ZF == 0 then PC = target
Operation:  if FREG.ZF == 0 then PC <- target
Example:  JNZ R3 ; Jump to address R3 if ZF == 0 (non-zero)
```

Configuration and Operands



Octet 0: Opcode 01011, bits $[2:0]$ (MOD): 000 (16-bit Imm address, 3 bytes), 001 (indirect via Rs, 2 bytes), 010 (24-bit via PTREG). Jump takes place if Zero Flag FREG.ZF = 0.

Flags Affected (FREG)

Preserves all status flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Fast-path condition evaluation in microcode.

TBD markers denote configurable hardware options.

JZ

Conditional Branch on Zero

01100
Branch

Mnemonic	JZ
Opcode	01100 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Branch
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Conditional Branch on Zero.

Architectural Semantics

```
Syntax:  JZ target(IMM|REG) : if FREG.ZF == 1 then PC = target
Operation:  if FREG.ZF == 1 then PC <- target
Example:  JZ R4 ; Jump to address R4 if ZF == 1 (zero)
```

Configuration and Operands



Octet 0: Opcode 01100, bits $[2:0]$ (MOD): 000 (16-bit Imm address, 3 bytes), 001 (indirect via Rs, 2 bytes), 010 (24-bit via PTREG). Jump takes place if Zero Flag FREG.ZF = 1.

Flags Affected (FREG)

Preserves all status flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Optimized for string matching and search algorithms.

TBD markers denote configurable hardware options.

AND

Bitwise Logical AND

01101
Logic

Mnemonic	AND
Opcode	01101 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

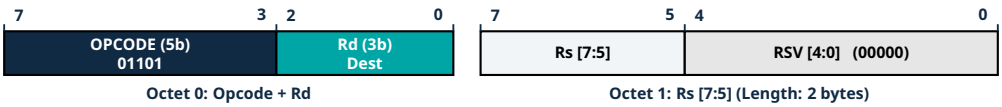
Instruction Purpose

Bitwise Logical AND.

Architectural Semantics

```
Syntax:  AND arg0(REG), arg1(REG) : arg0 = arg0 & arg1
Operation:  arg0 <- arg0 AND arg1
Example:  AND R0, R1 ; R0 = R0 & R1
```

Configuration and Operands



Octet 0: Opcode 01101, bits $[2:0]$ select destination register Rd (R0–R7, 1st argument).
Octet 1: Bits $[7:5]$ select source register Rs (R0–R7); bits $[4:0]$ are reserved (00000_2). Instruction length: 2 bytes.

Flags Affected (FREG)

Updates Zero Flag (ZF); resets overflow status.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Essential for bitmask extraction and masking.

OR

Bitwise Logical OR

01110
Logic

Mnemonic	OR
Opcode	01110 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

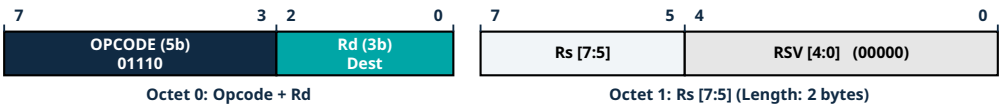
Instruction Purpose

Bitwise Logical OR.

Architectural Semantics

```
Syntax:  OR arg0(REG), arg1(REG) : arg0 = arg0 | arg1
Operation:  arg0 <- arg0 OR arg1
Example:  OR R0, R1 ; R0 = R0 | R1
```

Configuration and Operands



Octet 0: Opcode 01110, bits $[2:0]$ select destination register Rd (R0–R7, 1st argument).
Octet 1: Bits $[7:5]$ select source register Rs (R0–R7); bits $[4:0]$ are reserved (00000_2). Instruction length: 2 bytes.

Flags Affected (FREG)

Updates ZF; carry/overflow deterministically cleared.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Rapid combination of control bitfields.

TBD markers denote configurable hardware options.

NAND

Bitwise Logical NAND

01111
Logic

Mnemonic	NAND
Opcode	01111 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

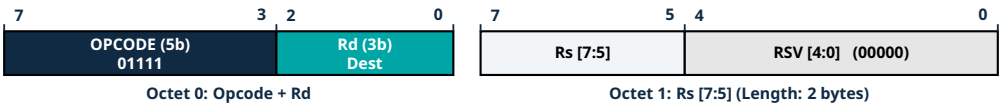
Instruction Purpose

Bitwise Logical NAND.

Architectural Semantics

```
Syntax:  NAND arg0(REG), arg1(REG) : arg0 = ~(arg0 & arg1)
Operation:  arg0 <- NOT (arg0 AND arg1)
Example:  NAND R0, R1 ; R0 = ~(R0 & R1)
```

Configuration and Operands



Octet 0: Opcode 01111, bits $[2:0]$ select destination register Rd (R0–R7, 1st argument).
Octet 1: Bits $[7:5]$ select source register Rs (R0–R7); bits $[4:0]$ are reserved (00000_2). Length: 2 bytes.

Flags Affected (FREG)

Updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Enables compact implementation of custom Boolean functions.

TBD markers denote configurable hardware options.

NOR

Bitwise Logical NOR

10000
Logic

Mnemonic	NOR
Opcode	10000 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

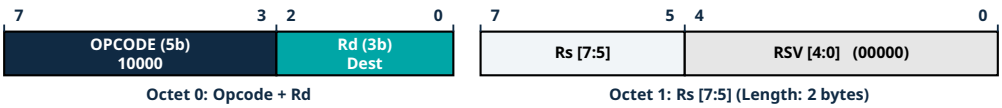
Instruction Purpose

Bitwise Logical NOR.

Architectural Semantics

```
Syntax:  NOR arg0(REG), arg1(REG) : arg0 = ~(arg0 | arg1)
Operation:  arg0 <- NOT (arg0 OR arg1)
Example:  NOR R0, R1 ; R0 = ~(R0 | R1)
```

Configuration and Operands



Octet 0: Opcode 10000, bits $[2:0]$ select destination register Rd (R0–R7, 1st argument).
Octet 1: Bits $[7:5]$ select source register Rs (R0–R7); bits $[4:0]$ are reserved (00000_2). Length: 2 bytes.

Flags Affected (FREG)

Updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Broadens fast bitwise logic transformations.

TBD markers denote configurable hardware options.

NOT

Bitwise Inversion (One's Complement)

10001
Logic

Mnemonic	NOT
Opcode	10001 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Bitwise Inversion (One's Complement).

Architectural Semantics

```
Syntax:  NOT arg0(REG) : arg0 = ~arg0
Operation:  arg0 <- NOT arg0
Example:  NOT R0 ; R0 = ~R0 (1-byte instruction)
```

Configuration and Operands



Single-byte instruction (length: 1 byte). Octet 0: Bits $[7:3]$ contain opcode 10001, bits $[2:0]$ directly encode target register Rd (R0–R7). No second byte is fetched from memory.

Flags Affected (FREG)

Updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Executes in minimal microcycles without fetching a second byte.

TBD markers denote configurable hardware options.

XOR

Bitwise Exclusive OR

10010
Logic

Mnemonic	XOR
Opcode	10010 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Bitwise Exclusive OR.

Architectural Semantics

```
Syntax:  XOR arg0(REG), arg1(REG) : arg0 = arg0 ^ arg1
Operation:  arg0 <- arg0 XOR arg1
Example:  XOR R0, R1 ; R0 = R0 ^ R1 (XOR R0, R0 clears R0)
```

Configuration and Operands



Octet 0: Opcode 10010, bits $[2:0]$ select destination register Rd (R0–R7, 1st argument).
Octet 1: Bits $[7:5]$ select source register Rs (R0–R7); bits $[4:0]$ are reserved (00000_2). Instruction length: 2 bytes.

Flags Affected (FREG)

Updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Foundational for parity, CRC, and cryptographic ciphers.

TBD markers denote configurable hardware options.

XNOR

Bitwise Equivalence

10011
Logic

Mnemonic	XNOR
Opcode	10011 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

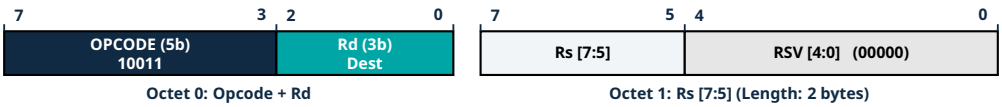
Instruction Purpose

Bitwise Equivalence.

Architectural Semantics

```
Syntax:  XNOR arg0(REG), arg1(REG) : arg0 = ~(arg0 ^ arg1)
Operation:  arg0 <- NOT (arg0 XOR arg1)
Example:  XNOR R0, R1 ; R0 = ~(R0 ^ R1)
```

Configuration and Operands



Octet 0: Opcode 10011, bits $[2:0]$ select destination register Rd (R0–R7, 1st argument).
Octet 1: Bits $[7:5]$ select source register Rs (R0–R7); bits $[4:0]$ are reserved (00000_2). Length: 2 bytes.

Flags Affected (FREG)

Updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Fast parallel mask equivalence validation.

TBD markers denote configurable hardware options.

INC

Atomic Increment by 1

10100
Arithmetic

Mnemonic	INC
Opcode	10100 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Atomic Increment by 1.

Architectural Semantics

```
Syntax:  INC arg0(REG) : arg0 = arg0 + 1
Operation:  arg0 <- arg0 + 1
Example:  INC R0 ; R0 = R0 + 1 (1-byte instruction)
```

Configuration and Operands



Octet 0 (1 byte, 0 extra bytes)

Single-byte instruction (length: 1 byte). Octet 0: Bits $[7:3]$ contain opcode 10100, bits $[2:0]$ directly encode target register Rd (R0–R7). No second byte is fetched from memory.

Flags Affected (FREG)

Updates ZF. Overflow is tracked by ALU.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Fast loop iterator step executed in minimal microcycles without extra fetch.

TBD markers denote configurable hardware options.

DEC

Atomic Decrement by 1

10101
Arithmetic

Mnemonic	DEC
Opcode	10101 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Atomic Decrement by 1.

Architectural Semantics

```
Syntax:  DEC arg0(REG) : arg0 = arg0 - 1
Operation:  arg0 <- arg0 - 1
Example:  DEC R0 ; R0 = R0 - 1 (1-byte instruction)
```

Configuration and Operands



Single-byte instruction (length: 1 byte). Octet 0: Bits $[7:3]$ contain opcode 10100, bits $[2:0]$ directly encode target register Rd (R0–R7). No second byte is fetched from memory.

Flags Affected (FREG)

Updates ZF. Borrow is tracked by ALU.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Compact loop termination construct without extra fetch.

TBD markers denote configurable hardware options.

PUSH

Push Word onto Execution Stack

10110
Stack

Mnemonic	PUSH
Opcode	10110 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Stack
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

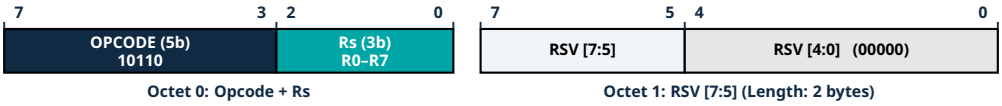
Instruction Purpose

Push Word onto Execution Stack.

Architectural Semantics

```
Syntax:  PUSH arg0(REG) : SREG -= 2, MEM[SREG] = arg0
Operation:  SREG <- SREG - 2, MEM[SREG] <- arg0
Example:  PUSH R0 ; Push word R0 onto execution stack
```

Configuration and Operands



Octet 0: Opcode 10110, bits $[2:0]$ directly designate source register Rs (R0–R7, 1st argument).
Octet 1: Bits $[7:0]$ are reserved (00000000_2). Instruction length: 2 bytes.

Flags Affected (FREG)

Preserves arithmetic flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Hardware-assisted subroutine and context preservation.

TBD markers denote configurable hardware options.

POP

Pop Word from Execution Stack

10111
Stack

Mnemonic	POP
Opcode	10111 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Stack
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

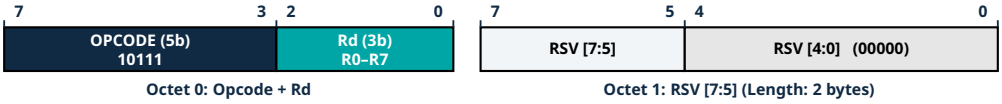
Instruction Purpose

Pop Word from Execution Stack.

Architectural Semantics

```
Syntax:  POP arg0(REG) : arg0 = MEM[SREG], SREG += 2
Operation:  arg0 <- MEM[SREG], SREG <- SREG + 2
Example:  POP R0 ; Pop word from top of stack into R0
```

Configuration and Operands



Octet 0: Opcode 10111, bits $[2:0]$ directly designate destination register Rd (R0–R7, 1st argument).
Octet 1: Bits $[7:0]$ are reserved (00000000_2). Instruction length: 2 bytes.

Flags Affected (FREG)

Preserves arithmetic flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Symmetric restoration of execution contexts.

BSL

Iterative Hardware Shift Left

11000
Shift

Mnemonic	BSL
Opcode	11000 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Shift
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

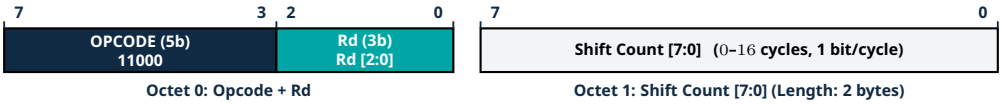
Instruction Purpose

Iterative Hardware Shift Left.

Architectural Semantics

```
Syntax:  BSL arg0(REG), arg1(IMM) : arg0 = arg0 « arg1
Encoding:  Octet 0:  [7:3]=11000, [2:0]=Rd (target register R0-R7)
           Octet 1:  [7:0]=shift (shift amount 0-16)
Operation:  Sequential clocked shift of register Rd left by shift bit
positions:
Hardware loop (1 cycle per bit, ≈ 300 transistors):
For each clock cycle t ∈ [1..shift]:
MSB bit 15 is displaced (last_displaced <- bit 15);
If SMOD = 1:  bit 15 wraps into LSB bit 0 (circular rotation);
If SMOD = 0:  LSB bit 0 receives bit from FREG.DBIT (0 or 1);
Capture:  FREG.DBIT <- last_displaced (last bit displaced out of Rd).
Example:  BSL R0, 4 ; Shift R0 left by 4 bits (takes 4 shift clock cycles)
```

Configuration and Operands



Octet 0: Opcode 11000, bits $[2:0]$ directly designate target register Rd (R0–R7).
Octet 1: Full byte ([7:0]) defines shift count shift (0–16). Hardware iterative D-flip-flop chain. Length: 2 bytes.

Flags Affected (FREG)

Last displaced bit latched into FREG.DBIT (bit 13). Zero flag FREG.ZF is set if result is zero. Controlled by FREG.SMOD (bit 14) and FREG.DBIT (bit 13).

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Iterative D-flip-flop shift register: costs only ≈ 300 transistors instead of $\approx 30\,000$ for a single-cycle matrix barrel shifter. Shifting by 16 bits takes 15 more cycles than shifting by 1 bit.

TBD markers denote configurable hardware options.

BSR

Iterative Hardware Shift Right

11001
Shift

Mnemonic	BSR
Opcode	11001 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Shift
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

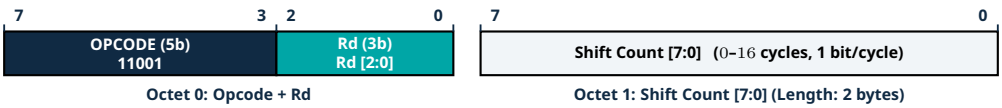
Instruction Purpose

Iterative Hardware Shift Right.

Architectural Semantics

```
Syntax:  BSR arg0(REG), arg1(IMM) : arg0 = arg0 » arg1
Encoding:  Octet 0:  [7:3]=11001, [2:0]=Rd (target register R0–R7)
           Octet 1:  [7:0]=shift (shift amount 0–16)
Operation:  Sequential clocked shift of register Rd right by shift bit
positions:
Hardware loop (1 cycle per bit, ≈ 300 transistors):
For each clock cycle t ∈ [1..shift]:
LSB bit 0 is displaced (last_displaced <- bit 0);
If SMOD = 1:  bit 0 wraps into MSB bit 15 (circular rotation);
If SMOD = 0:  MSB bit 15 receives bit from FREG.DBIT (0 or 1);
Capture:  FREG.DBIT <- last_displaced (last bit displaced out of Rd).
Example:  BSR R0, 4 ; Shift R0 right by 4 bits (takes 4 shift clock cycles)
```

Configuration and Operands



Octet 0: Opcode 11001, bits $[2:0]$ directly designate target register Rd (R0–R7).
Octet 1: Full byte ([7:0]) defines shift count shift (0–16). Hardware iterative D-flip-flop chain. Length: 2 bytes.

Flags Affected (FREG)

Last displaced bit latched into FREG.DBIT (bit 13). Zero flag FREG.ZF is set if result is zero. Controlled by FREG.SMOD (bit 14) and FREG.DBIT (bit 13).

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Iterative D-flip-flop shift register: costs only ≈ 300 transistors instead of $\approx 30\,000$ for a single-cycle matrix barrel shifter. Shifting by 16 bits takes 15 more cycles than shifting by 1 bit.

TBD markers denote configurable hardware options.

MSP

Manage Special Registers (FREG, PTREG, SREG, VEC)

11010
System

Mnemonic	MSP
Opcode	11010 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	System
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Manage Special Registers (FREG, PTREG, SREG, VEC).

Architectural Semantics

```
Syntax:  MSP sreg, [op], args :  special register operations (2-6 bytes)
Operation:  Direct Load (Bit 10 = 0):
sreg=00 (FREG): FREG <- Rs[15:0] or imm16 / [addr16] / [$ptreg]
sreg=01 (PTREG): 32-bit pointer PTREG (4 GB): from GPR, imm32, [$ptreg],
[addr32]
sreg=10 (SREG): 32-bit stack pointer SREG: from GPR, imm32, [$ptreg],
[addr32]
sreg=11 (VEC): 32-bit vector for IRQ ($ir0..$ir15) or zone ($pv0..$pv15)
Modify/Operate (Bit 10 = 1):
sreg=00 (FREG): FREG <- FREG ⊙ Mask {AND, OR, XOR, XNOR}
sreg=01 (PTREG): PTREG <- PTREG ± Offset {ADD, SUB}
sreg=10 (SREG): SREG <- SREG ± Offset {ADD, SUB}
Example:  MSP.ld $ir0, 0x00010000 ; Install IRQ0 handler vector (6 bytes)
MSP.ld $pv0, 0x00008000 ; Privilege Zone 0 start boundary
MSP.ld $pv1, 0x00008FFF ; Privilege Zone 0 end boundary
MSP.ld $ptreg, [$ptreg] ; Native single-cycle dereference (2 bytes)
MSP.or $freg, 0x0200 ; Enable global interrupts MIE
```

Configuration and Operands



Octet 0: Opcode 11010. Bit $[10]$ sets mode (0 = Load, 1 = Modify). Bits $[9:8]$ select register (00 = FREG, 01 = PTREG, 10 = SREG).
Octet 1: Under load ($[10] = 0$), selects Rs for FREG or pair Rs_hi, Rs_lo for PTREG/SREG. Under

modification ($[10] = 1$), for FREG bits $[7:6]$ encode operation (AND, OR, XOR, XNOR) and bits $[5:3]$ select mask register Rs; for PTREG and SREG, bit $[7]$ selects operation (0=ADD, 1=SUB) and bits $[6:4]$ select single offset register Rs. Instruction length: 2 bytes.

Flags Affected (FREG)

FREG flags modified directly or by mask. PTREG and SREG arithmetic updates CF carry/borrow via the ALU.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Universal privileged control primitive for system architectures.

TBD markers denote configurable hardware options.

MOD

Integer Modulo Operation

11011
Arithmetic

Mnemonic	MOD
Opcode	11011 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

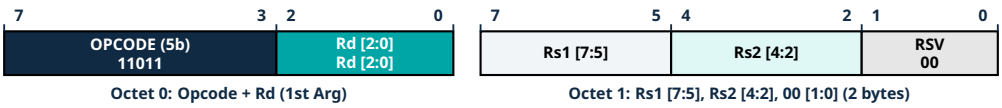
Instruction Purpose

Integer Modulo Operation.

Architectural Semantics

```
Syntax:  MOD arg0, arg1, arg2 :  arg0 = arg1 % arg2
Operation:  dst <- arg1 % arg2
Example:  MOD R0, R1, R2 ; R0 = R1 % R2 (integer remainder)
```

Configuration and Operands



Octet 0: Opcode 11011, bits $[2:0]$ designate destination register Rd (R0–R7, 1st argument).
Octet 1: Bits $[7:5]$ designate dividend Rs1 (R0–R7); bits $[4:2]$ designate divisor Rs2 (R0–R7); bits $[1:0]$ are reserved (00). Length: 2 bytes.

Flags Affected (FREG)

Division by zero asserts hardware error flag EM (Error Math) without core lockup; updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Calculates the integer remainder (modulo) of dividing Rs1 by Rs2 and stores the result directly into destination register Rd without overwriting temporary accumulator registers.

TBD markers denote configurable hardware options.

INTR

Software Interrupt via 3-Bit Vector Index (1 Byte)

11100
System

Mnemonic	INTR
Opcode	11100 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	System
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Software Interrupt via 3-Bit Vector Index (1 Byte).

Architectural Semantics

```
Syntax:  INTR vec3 :  invoke software interrupt vector vec3 (0..7)
Operation:  SREG <- SREG - 2; MEM16[SREG] <- PC_next
SREG <- SREG - 2; MEM16[SREG] <- FREG
FREG.INTR <- 1; FREG.MIE <- 0; PC <- VECTORS[vec3]
Example:  INTR 3 ; Software interrupt via vector #3 (opcode 0xE3)
```

Configuration and Operands



Octet 0 (1 byte, 0 extra bytes)

Octet 0: Opcode 11100, lowest 3 bits $[2:0]$ directly specify the interrupt vector index vec3 (0–7) from the internal vector table in the processor. Length: 1 byte.

Flags Affected (FREG)

Sets active interrupt flag FREG.INTR (bit 15) to 1 and clears FREG.MIE (bit 9) to 0. Previous FREG and return address are saved on the stack.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Single-byte software interrupt instruction (system call or debug trap) for immediate jump to a handler from the processor’s internal vector stack/table.

TBD markers denote configurable hardware options.

RSV29

Architectural Extension Slot

11101
Reserved

Mnemonic	RSV29
Opcode	11101 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Reserved
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Architectural Extension Slot.

Architectural Semantics

```
Syntax:  RSV29 :  reserved
Operation:  Reserved for LIM architecture hardware extensions
```

Configuration and Operands



Octet 0 (1 byte, 0 extra bytes)

Reserved for future LIM ISA revisions. Length: 1 byte.

Flags Affected (FREG)

Undefined.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Reserved.

TBD markers denote configurable hardware options.

RSV30

Architectural Extension Slot

11110
Reserved

Mnemonic	RSV30
Opcode	11110 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Reserved
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Architectural Extension Slot.

Architectural Semantics

```
Syntax:  RSV30 :  reserved
Operation:  Reserved for LIM architecture hardware extensions
```

Configuration and Operands



Octet 0 (1 byte, 0 extra bytes)

Reserved for future LIM ISA revisions. Length: 1 byte.

Flags Affected (FREG)

Undefined.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Reserved.

TBD markers denote configurable hardware options.

RSV31

Architectural Extension Slot

11111
Reserved

Mnemonic	RSV31
Opcode	11111 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Reserved
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Architectural Extension Slot.

Architectural Semantics

```
Syntax:  RSV31 :  reserved
Operation:  Reserved for LIM architecture hardware extensions
```

Configuration and Operands



Octet 0 (1 byte, 0 extra bytes)

Reserved for future LIM ISA revisions. Length: 1 byte.

Flags Affected (FREG)

Undefined.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

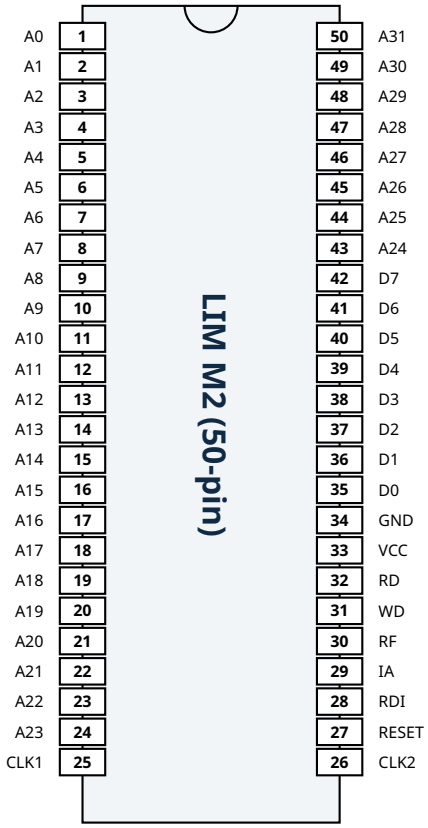
Implementation Notes

Reserved.

TBD markers denote configurable hardware options.

7 Physical Package and Pinout (50-Pin Form Factor)

The processor is packaged in a high-reliability 50-pin DIP-50 / QFP-52 enclosure featuring a native 32-bit address bus and an 8-bit (1-byte) external data bus:



7.1 Pin Descriptions

Signal	Direction	Functional Role
A0–A31	Output / Tri-State	32-bit native address bus (flat physical access to 4 GB memory without paging/banking).
D0–D7	Bidirectional	8-bit (1-byte) external memory and peripheral data interface.
CLK1, CLK2	Input	Two-phase non-overlapping clock synchronization inputs.
RESET	Input	Active-low master hardware reset signal.
RDI	Input	Raw Direct Interrupt: priority hardware interrupt request line.
IA	Output	Interruption Accepted / Interrupt Acknowledge: informs peripherals of accepted request.
RF	Input	Ready Flag: external slave memory/peripheral handshake acknowledge.
RD, WD	Output	Bus Read Enable and Write Enable strobes.
VCC, GND	Power	Primary digital power supply (+3.3 V / +5 V nominal) and logic ground.

8 Interrupt Subsystem and Vector Processing

The LIM M2 interrupt subsystem is architected to guarantee deterministic low-latency real-time response, bus contention immunity, and atomic context switching. The architecture integrates external hardware service requests and internal software traps into a unified 16-vector hierarchy governed by a cycle-accurate hardware handshake.

8.1 Unified 16-Vector Interrupt Space

The processor provides exactly 16 system interrupt vectors (indices 0–15), which are orthogonally shared between peripheral devices and internal CPU events:

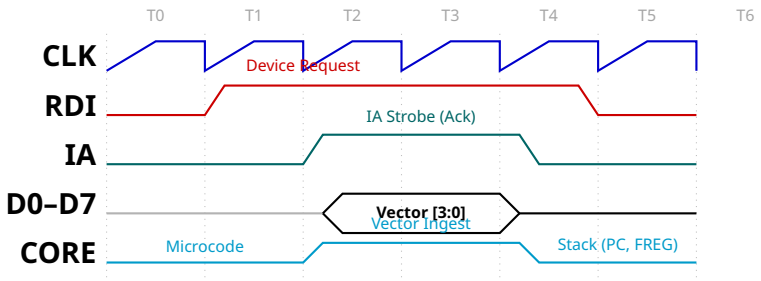
Table 3: LIM M2 Unified 16-Vector Interrupt Table

Vector	Code	Source / Type	Functional Role
VEC 0	`0x00`	Hardware / Reset	Power-on Reset: primary core and register initialization.
VEC 1	`0x01`	Hardware (NMI)	Non-Maskable Interrupt: power-fail warning, critical hardware alert.
VEC 2	`0x02`	Software Trap	Arithmetic Exception (division by zero, `FREG.EM` flag).
VEC 3	`0x03`	Software Trap	Operating System System Call (Syscall / API trap dispatcher).
VEC 4	`0x04`	Hardware / Periph	System Interval Timer / High-resolution scheduler tick.
VEC 5	`0x05`	Hardware / Periph	Serial Communications Interface (UART RX/TX service).
VEC 6	`0x06`	Hardware / Periph	Direct Memory Access Controller (DMA Transfer Complete).
VEC 7	`0x07`	Hardware / Periph	External Interrupt Controller / Expansion bus bridge.
VEC 8–15	`0x08–0x0F`	Shared (HW / SW)	User hardware interrupts and software-defined handlers.

8.2 Hardware Handshake Protocol (RDI → IA → D0–D7)

For robust peripheral interfacing, the processor implements a 4-phase hardware handshake cycle that completely eliminates CPU polling overhead:

1. **Request Phase (RDI):** A peripheral device asserts an active-high level on the `RDI` (*Raw Direct Interrupt*) pin. The core samples `RDI` at atomic micro-op boundaries.
2. **Arbitration Phase (Mask Check):** If interrupts are enabled (`FREG.MIE` = 1), the CPU commits the current datapath micro-operation and initiates the interrupt sequence.
3. **Acknowledge Phase (IA Strobe):** The core asserts an active-high strobe on the `IA` (*Interruption Accepted / Interrupt Acknowledge*, Pin 26) output. This informs all slave devices on the bus that the core is ready to ingest the interrupt vector index.
4. **Vector Transfer Phase (D0–D7):** Concurrently with asserting `IA`, the processor switches the external bidirectional data bus `D0–D7` to input mode and activates the `RD` line. The requesting peripheral drives its 8-bit vector ID onto `D0–D7` (lower 4 bits address vectors 0–15).
5. **Latch and Dispatch Phase:** The CPU latches the vector ID from `D0–D7`, deasserts `IA`, and advances to context preservation.



8.3 Context Preservation on 24-Bit Stack and Handler Entry

Interrupt service entry is permitted strictly when the CPU is not already executing an interrupt handler (FREG.INTR = 0) and master interrupts are unmasked (FREG.MIE = 1). When FREG.INTR = 1, subsequent hardware interrupt requests are locked out by hardware.

Upon interrupt acceptance, the core executes an atomic state preservation sequence:

1. **Program Counter Preservation (PC):** The current instruction pointer is pushed onto the 24-bit execution stack via SREG:

$$\text{SREG} \leftarrow \text{SREG} - 2, \quad \text{Memory}[\text{SREG}] \leftarrow \text{PC}_{15:0}$$

2. **Status and Flags Preservation (FREG):** The complete status register (with FREG.INTR=0) is pushed onto the stack:

$$\text{SREG} \leftarrow \text{SREG} - 2, \quad \text{Memory}[\text{SREG}] \leftarrow \text{FREG}$$

3. **Nested Interrupt Lockout:** The core asserts the in-interrupt flag FREG.INTR $\leftarrow$ 1 (bit 15) and clears the mask bit FREG.MIE $\leftarrow$ 0, precluding nested re-entrancy and race conditions.
4. **Vector Dispatch:** The handler address is indexed from the vector table using the ingested vector ID and loaded into PC.

8.4 Return from Interrupt

Exiting the interrupt service routine restores the preempted execution context:

1. The saved flags and processor modes are restored from the stack:

$$\text{FREG} \leftarrow \text{Memory}[\text{SREG}], \quad \text{SREG} \leftarrow \text{SREG} + 2$$

   (Restoring FREG automatically clears FREG.INTR back to 0, lifting the lockout, and restores FREG.MIE to its previous state).
2. The return address is popped back into the program counter:

$$\text{PC} \leftarrow \text{Memory}[\text{SREG}], \quad \text{SREG} \leftarrow \text{SREG} + 2$$

3. Normal instruction execution seamlessly resumes on the subsequent clock cycle with zero data corruption.

9 LIM M2 Assembly Language and Compiler (limasm.py)

The two-pass assembler compiler LIM2/sim/limasm.py translates LIM M2 assembly source files (.asm / .lasm) simultaneously into raw binary images (.bin) and hexadecimal files (.hex).

9.1 Register Naming and Operand Size Suffixes

- **General-Purpose Registers (16-bit):** denoted with the $ prefix as $0–$7 (also accepts R0–R7).
- **Special System Registers:** $ptreg (24-bit pointer PTREG), $freg (16-bit flag/status register FREG), $sreg / $sp (24-bit stack pointer SREG), and $vec0–$vec7.
- **Size Suffixes (.b / .w / .d):** instructions where operand size is specified use XXX.b (byte, 8-bit), XXX.w (word, 16-bit, default), or XXX.d (doubleword, 24/32-bit).

9.2 Register Arithmetic, Unary Ops, and Memory Access (LMR, LRM)

- **Register-Only ALU and 1-Byte Unary Instructions:**

```
ADD $0, $1, $2      ; $0 = $1 + $2 (or ADD R0, R1, R1; same for SUB, MUL, DIV, MOD
INC $0              ; $0 = $0 + 1  (1-byte instruction, or INC R0; same for DEC, N
```

- **LMR and LRM Addressing Modes:**
  1. **Bare number (no brackets)** — immediate value (SRC $= 0$ / DST $= 0$): LMR.w $0, 0x1234, LMR.b $1, 0x42.
  2. **Bracketed address [addr]** — direct memory reference (SRC $= 1$ / DST $= 1$): LMR.w $0, [0x1000], LRM.w $0, [0x1000].
  3. **Bracketed [$ptreg]** — 24-bit pointer via PTREG (SRC $= 2$ / DST $= 2$): LMR.w $0, [$ptreg], LRM.b $4, [$ptreg].
  4. **Register-pair pointer [$0:$1]** (SRC $= 3$ / DST $= 3$): LMR $0, [$0:$1] (or LMR, $0, [$0:$1]), LRM $0, [$0:$1].

9.3 Special Register Manipulation (MSP) and Multi-Dereferencing via PTREG

```
MSP.and $freg, 0x0001       ; Apply AND mask to flag register: FREG &= 0x0001
MSP.ld  $freg, 0x0000       ; Load immediate configuration into FREG
MSP.ld  [$ptreg], $0, $1    ; Load 24-bit pointer into PTREG from register pair ($0
MSP.ld  $ptreg, 0xA000      ; Load immediate address 0xA000 into PTREG
MSP.ld  $ptreg, [0xA000]    ; Load pointer from memory reference [0xA000] into PTRE

; Native multi-level pointer dereferencing via PTREG:
MSP.ld  $ptreg, 0xA000      ; Root pointer address
MSP.ld  $ptreg, [$ptreg]    ; 1st dereference: PTREG <- MEM[PTREG]
MSP.ld  $ptreg, [$ptreg]    ; 2nd dereference: PTREG <- MEM[PTREG]
MSP.ld  $ptreg, [$ptreg]    ; 3rd dereference: PTREG <- MEM[PTREG]
LMR.w   $0, [$ptreg]        ; Read final target value into $0
```

Compile to .bin and .hex: python3 LIM2/sim/limasm.py LIM2/sim/demo.asm -l.

10 Conclusion

The **LIM M2** architecture harmonizes the time-tested reliability of deterministic microprogramming with the modern performance of an entirely tri-state-free multiplexer datapath. By eliminating bus

contention, floating nodes, and capacitive decay, the LIM M2 guarantees exceptional signal fidelity and opens an unprecedented clock frequency envelope spanning from 200 MHz to the highest thresholds of modern silicon.